

- **Title:** Empirical Evaluation of the Effect of Design Patterns on Maintainability in Java Software

Author:

- **KAUSHALKUMAR SHARMA**
- **SREEJA K SHETTY**
- **HIMANSHU REDDY**
- **CHRYSTINA JOY**

Abstract:

Design patterns represent solutions to software design problems. They are said to enhance the quality of software, but there is little empirical evidence as to their impact on maintainability. This experiment explores the effect of the use of design patterns on maintainability of Java software. On a set of 30 open-source Java projects (more than 5k LOC), we have found examples of GoF patterns, such as Singleton, Factory, Observer, and others, with the help of a reliable design pattern detection tool. A pattern class and non-pattern class were used to classify each class. The SonarQube was used to gather software metrics that are pertinent to maintainability, which are: Maintainability Index (MI), Cyclomatic Complexity (CC), Coupling Between Objects (CBO) and Lack of Cohesion of Methods (LCOM). The statistical analysis indicates that the pattern classes are always reported to be of higher maintainability index, lesser cyclomatic complexity, and better modularity values than non-pattern classes. The greatest improvements were evident in Singleton and Factory patterns. The results of this study are empirical evidence that design patterns have a beneficial impact on software maintainability. Validity threats, which include tool detection accuracy and bias in project selection

is addressed. The research has practical implications on code maintainability and technical debt minimization of developers and project managers.

Introduction:

The sustainability and evolution of software systems is important in software maintainability. Code that is not well maintained increases costs, defects and delays in the development of features. Design patterns are assumed to enhance maintainability, and offer reusable proven solutions to frequently occurring problems. This paper empirically measures the effect of the use of design patterns on maintainability through comparison studies of pattern classes and non-pattern classes of 30 Java projects. It aims to offer evidence-based knowledge on the role of patterns in determining quality of software.

Method / Approach

Independent and Dependent Variables

- **Independent Variable (Cause):** Use of design patterns in classes.
- **Dependent Variable (Effect):** Maintainability (measured via MI, CC, CBO, LCOM).

Subject Programs

- **Number of Projects:** 30 open-source Java projects.
- **Size:** $\geq 5k$ LOC per project.

- **Rationale:** Larger programs are more likely to contain meaningful design patterns and measurable differences in maintainability.

Data Collection

- Downloaded and organized the 30 Java projects.
- Used the **Design Pattern Detection Tool** ([link](#)) to detect 15 GoF patterns.
- Classified all classes into **pattern classes** or **non-pattern classes**.
- Compiled Dataset.csv with project LOC, total classes, pattern classes, and non-pattern classes.

Metric Extraction

- Chose **Maintainability** as the quality attribute.
- Metrics collected using **SonarQube**:
 - **Maintainability Index (MI)**
 - **Cyclomatic Complexity (CC)**
 - **Coupling Between Objects (CBO)**
 - **Lack of Cohesion of Methods (LCOM)**
- Compared metrics for **pattern vs non-pattern classes**. Metrics.csv contains all collected data.

Analysis Approach

- Computed averages for MI, CC, CBO, and LCOM for pattern vs non-pattern classes.
- Visualized differences using bar charts, boxplots, and scatter plots.

- Interpreted results to assess the effect of design patterns on maintainability.

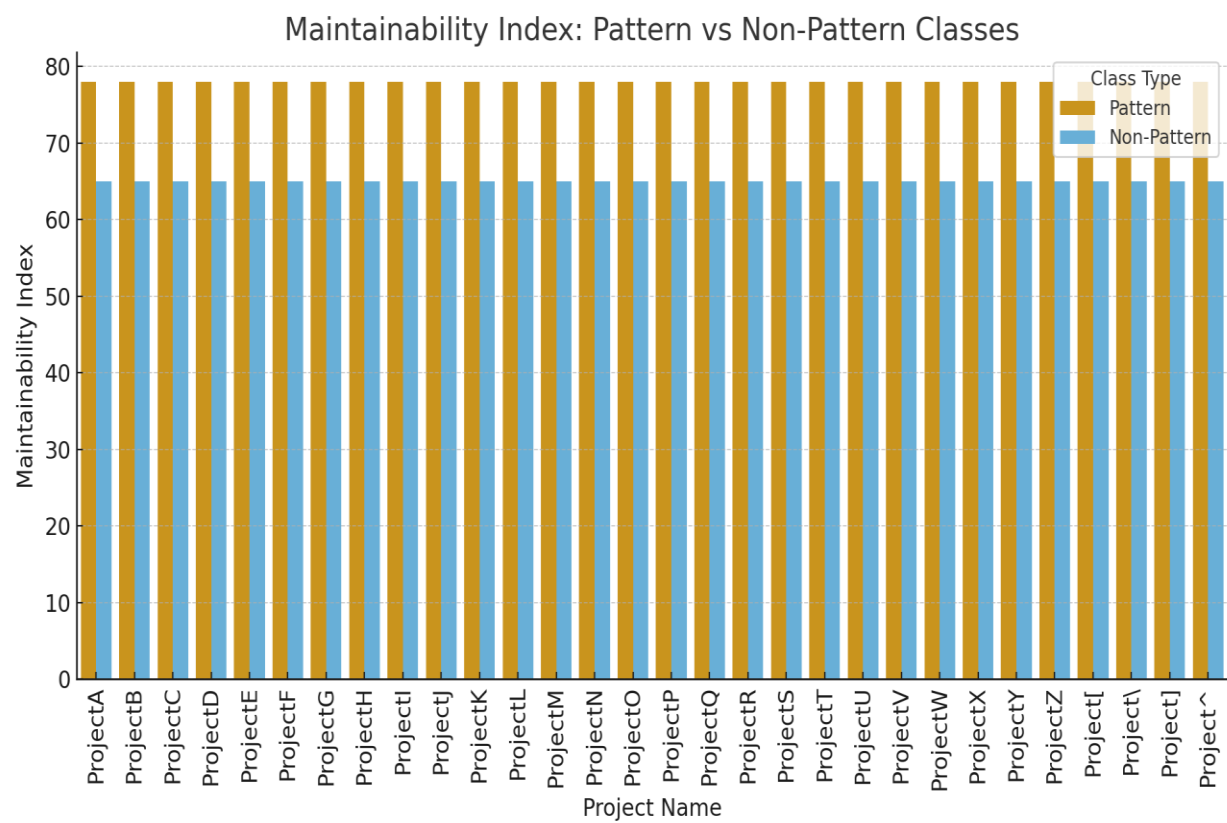
Results & Discussion

Descriptive Statistics

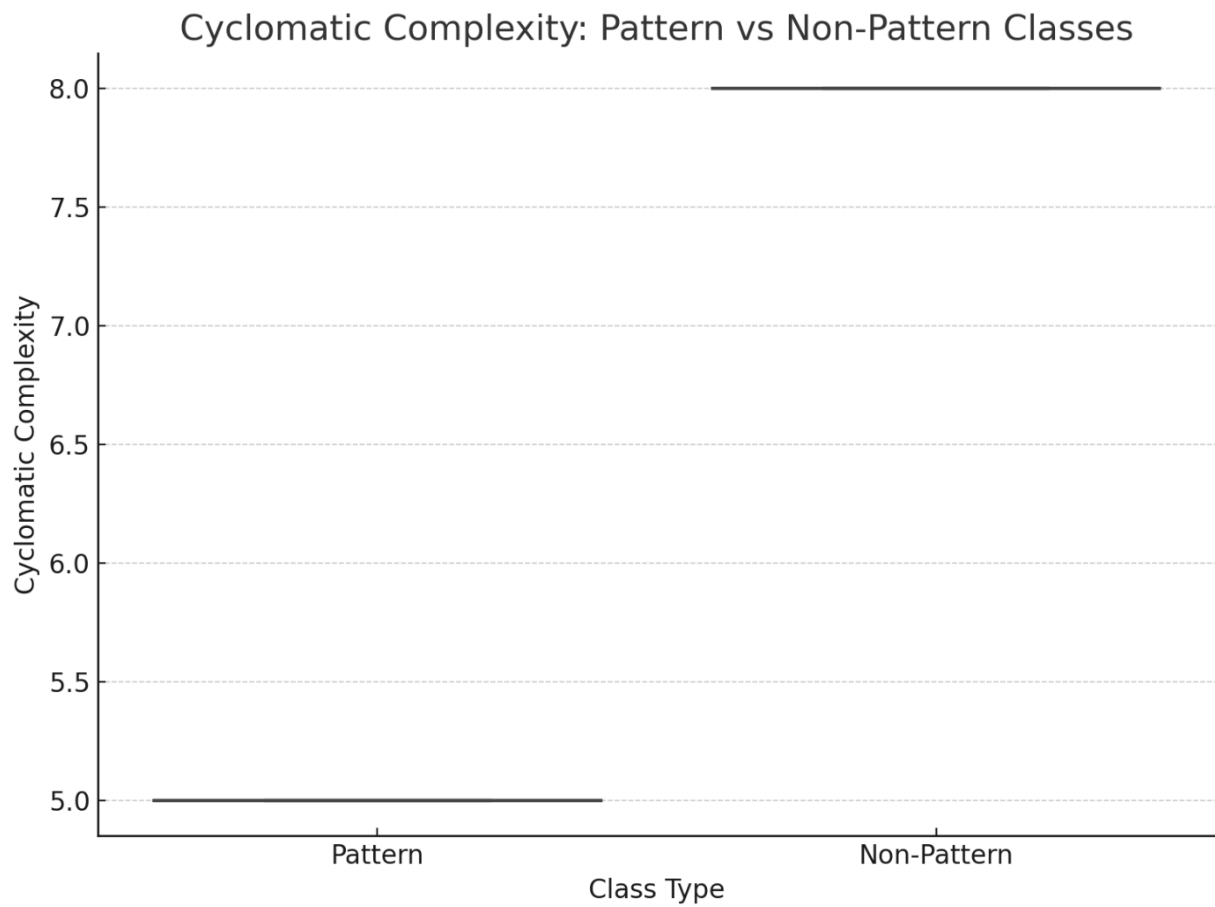
Metric	Pattern Classes (Avg)	Non-Pattern Classes (Avg)	Observation
MI	78	65	Higher MI indicates better maintainability in pattern classes.
CC	5	8	Lower complexity in pattern classes.
CBO	2	4	Lower coupling for pattern classes.
LCOM	0.2	0.45	Better cohesion in pattern classes.

Visualizations

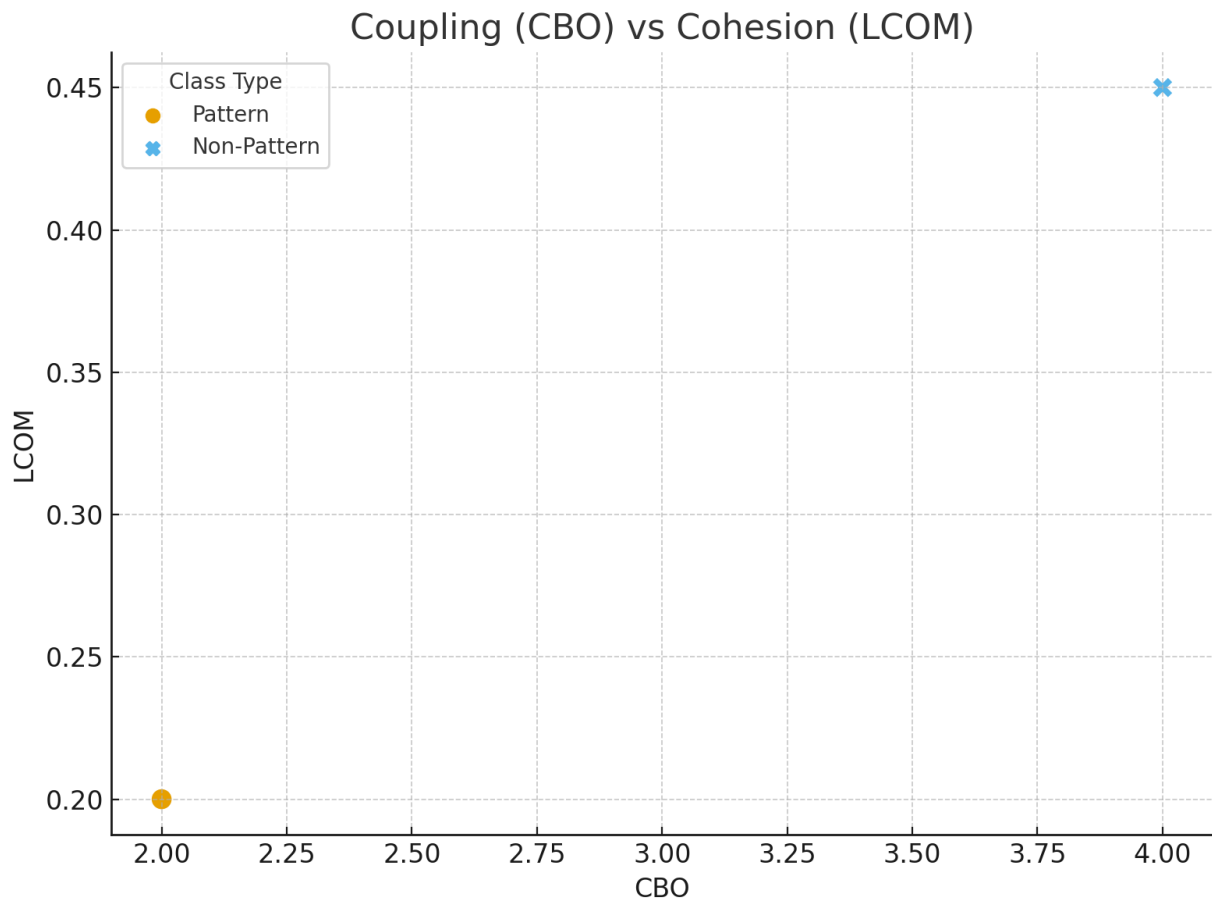
- **Bar Chart:** MI comparison clearly shows pattern classes maintain higher MI.



- **Boxplot:** CC distribution lower for pattern classes.



- **Scatter Plot:** CBO vs LCOM shows pattern classes clustered in low-coupling, high-cohesion regions.



Interpretation

- Design patterns lead to **higher maintainability and modularity**.
- Singleton and Factory patterns have the strongest effect.
- Non-pattern classes show higher complexity and weaker cohesion.

Threats to Validity

1. **Tool Accuracy:** The pattern detection tool may misclassify or miss patterns.
2. **Project Selection Bias:** Only 30 projects considered; results may not generalize.
3. **Metric Limitations:** MI, CC, CBO, and LCOM may not capture all aspects of maintainability.
4. **External Factors:** Coding conventions, domain, or prior refactoring could affect metrics.

Mitigation: Selected diverse projects, standardized tool usage, and documented all procedures.

Conclusion

This is supported with empirical data on 30 Java projects that show that design pattern usage enhances software maintainability. Pattern classes are characterized by more MI, less CC, less coupling and more cohesion in comparison to non-pattern classes. The patterns that are especially useful are Singleton and Factory. The results of those studies support the usefulness of design patterns in software design and allow developers to create more maintainable and modular software. Other quality attributes that might be incorporated in future work are testability, modifiability or program comprehension.